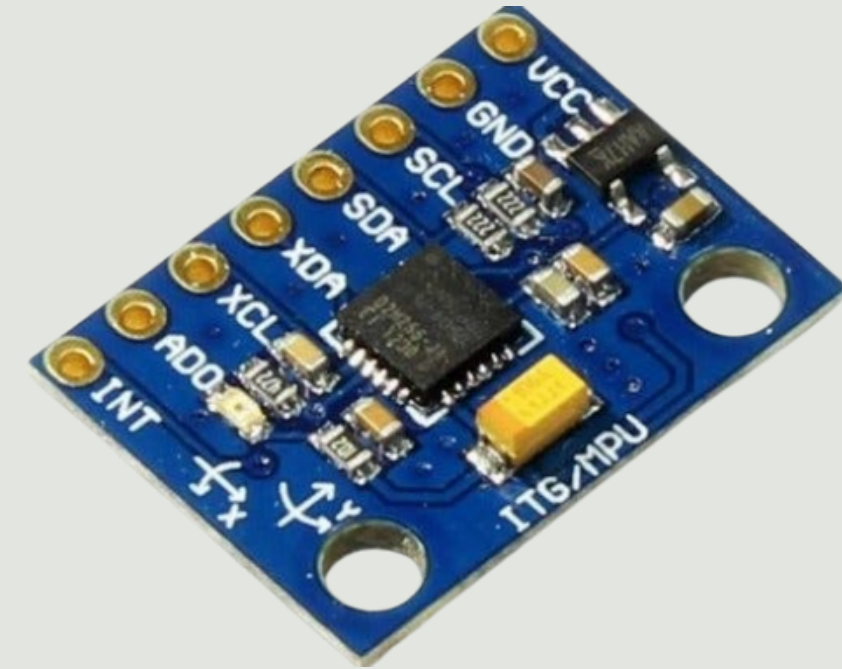


ROS2 BASED MOBILE ROBOT

PROBLEM STATEMENT

- Our problem statement is to create a barebone version of a bot which can localize itself and map its surroundings, hence we chose to do a simple ROS2 based Differential drive bot.
- It integrates data from multiple onboard sensors—such as IMU, motor encoders, LiDAR, and depth cameras—to estimate the robot's trajectory and build a spatial map in real-time.
- SLAM algorithm combines techniques from sensor fusion, estimation theory, and computer vision to achieve accurate localization without relying on GPS. This is the final goal we are trying to attain.
- It forms the backbone of autonomous navigation, robotic exploration, and smart mapping applications such as self-driving cars, drones, and mobile robots

IMU - SIGNAL CONDITIONING



Calibration → Madgwick Filter → Low Pass Filter

- Statistical analysis
- Static error
- Curve fitting
- sensor fusion
- dynamic error
- remove jitteryness from output

Madgwick Filter

Need: sensor fusion algorithm combines:

Gyroscope: Good for fast, short-term rotation.

Accelerometer: When the device is relatively still, it reliably measures the direction of gravity. This gives you a stable, long-term "down" vector, which can be used to figure out pitch and roll.

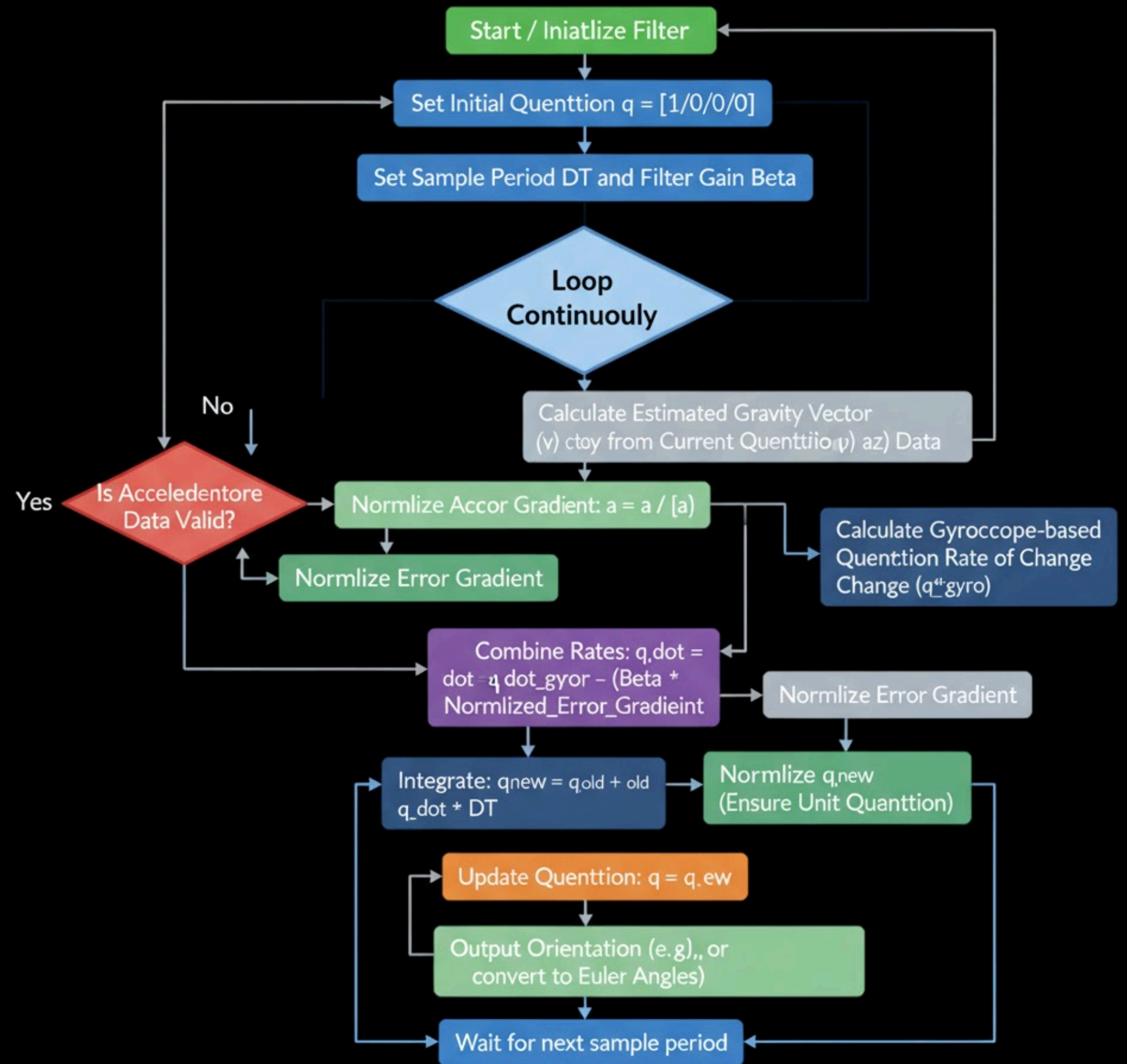
HOW MADGWICK FIXES THE PROBLEM



The Madgwick filter is a clever way to combine these sensors:

- It primarily trusts the gyroscope for its frame-to-frame orientation changes (the integration step).
- It simultaneously uses the accelerometer and/or magnetometer readings to calculate the "error" in the gyro's current orientation.
- It uses a mathematical technique (gradient descent) to find the "direction" to rotate the current orientation to correct this error.
- It applies this correction back to the gyroscope-based orientation, effectively "pulling" it back on track and eliminating the long-term drift.

ALGORITHM FLOWCHART



LIDAR - SIGNAL CONDITIONING

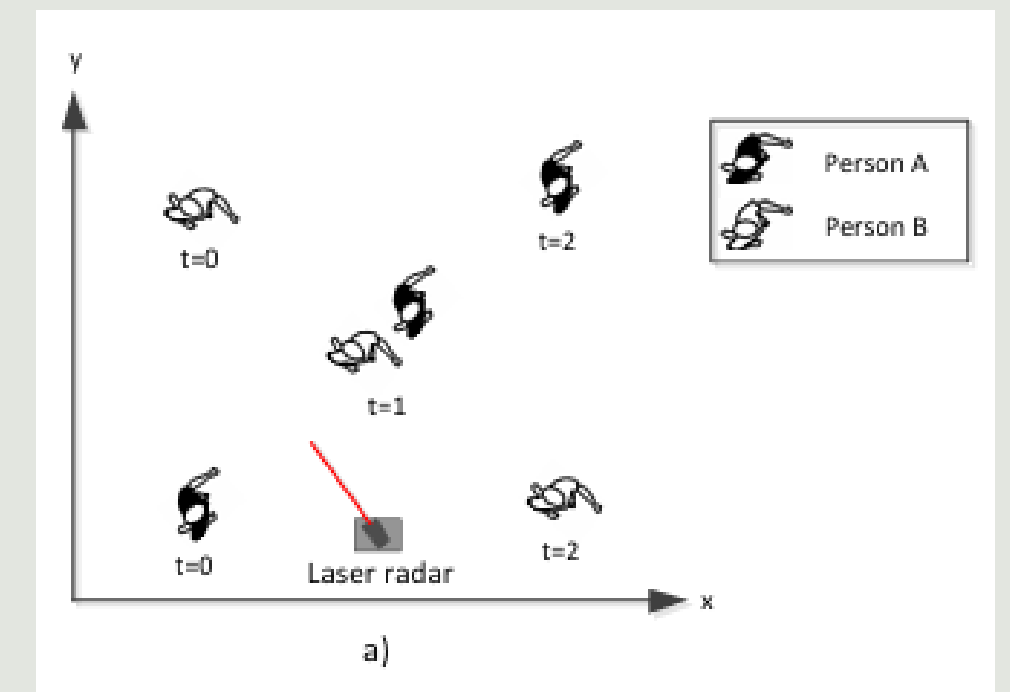
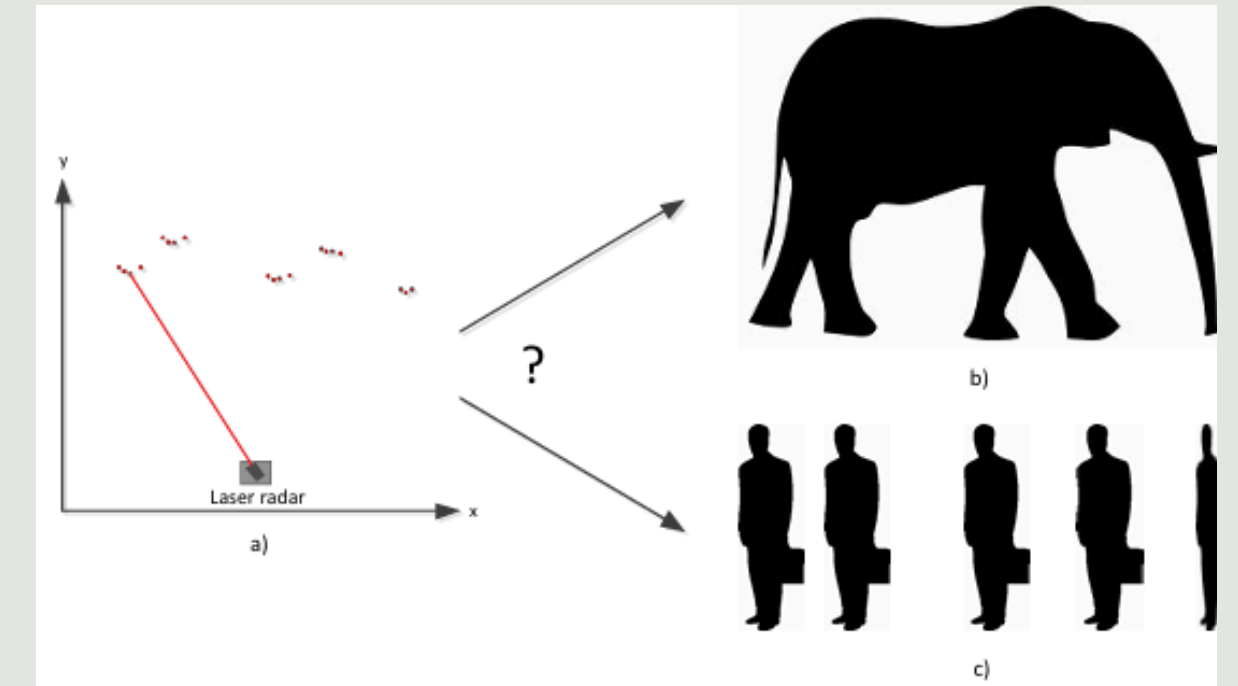
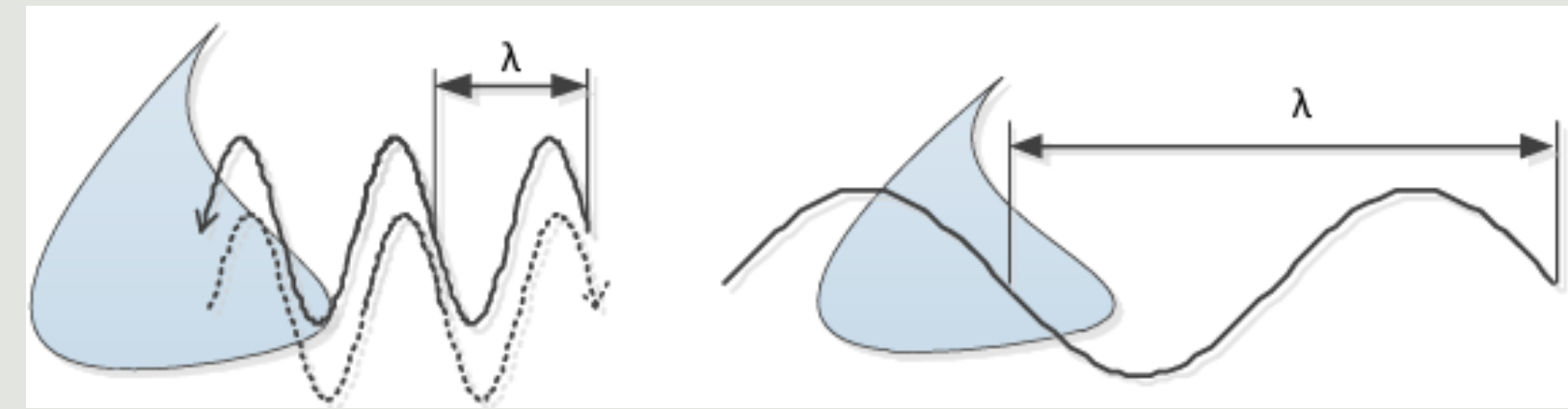
The Lidar is based on a laser distance meter which moves in the vertical axis to create an image of the environment.

This type of Lidar can have many applications where long range and high accuracy is needed, such as perimeter control of docks, airports and larger industrial complexes, as well as for instance finding faults with large industrial machinery.



PROBLEMS TO SOLVE WITH LIDAR

- Atmospheric Disturbances: Short laser wavelengths are highly prone to reflection from small particles (e.g., fog, rain, snow).
- Object Quantization: LIDAR scans produce variable numbers of data points per object, complicating target identification and classification. Since a LiDAR scan is essentially a projection from $\mathbb{R}^3$ to $\mathbb{R}^2$, how can we determine whether those returned points represent a single continuous object—or several distinct ones separated in space?
- Data association: Since a scanning laser system typically have a low frame rate, data association becomes a major problem when working on dynamical systems. E.g. how can you tell that two points at two different locations in space and time actually represent the same object?



CLUTTER SUPPRESSION FILTER

LiDAR returns not only reflections from real objects (vehicles, pedestrians, road edges) but also from unwanted particles like dust, fog, snow, or raindrops.

These false reflections are called clutter — they create noisy, unstable points in the LiDAR point cloud.

To maintain reliable object detection, a clutter suppression filter is applied in software after signal digitization.

- It analyzes each return point based on several physical and statistical parameters
- The filter evaluates these parameters and eliminates points that fail to meet the criteria, leaving only stable, high-confidence data.

Parameter	Meaning	Purpose
Amplitude / Intensity	Strength of reflected pulse	Low intensity → likely dust or fog
Spatial density	Number of nearby points	Real objects form dense clusters;
Temporal stability	Consistency across frames	Real objects persist; clutter flickers
Range validity	Physical limit of LiDAR	Out-of-range returns are discarded
SNR (Signal-to-Noise Ratio)	Amplitude vs background noise	Low SNR → unreliable reflection

PSUEDO CODE AND FLOWCHART

```
// Input: lidar_points[] = array of measured points  
(x, y, z, intensity, timestamp)  
// Output: clean_points[] = clutter-free points
```

```
for each point in lidar_points:
```

```
    // 1. Range check
```

```
    if (point.range > MAX_RANGE or point.range < MIN_RANGE)  
        continue // discard
```

```
    // 2. Intensity / SNR check
```

```
    if (point.intensity < INTENSITY_THRESHOLD)  
        continue // weak reflection, likely clutter
```

```
    // 3. Spatial density check
```

```
    neighbors = count_neighbors(point, radius = 0.2m)
```

```
    if (neighbors < 3)
```

```
        continue // isolated point = dust/snow
```

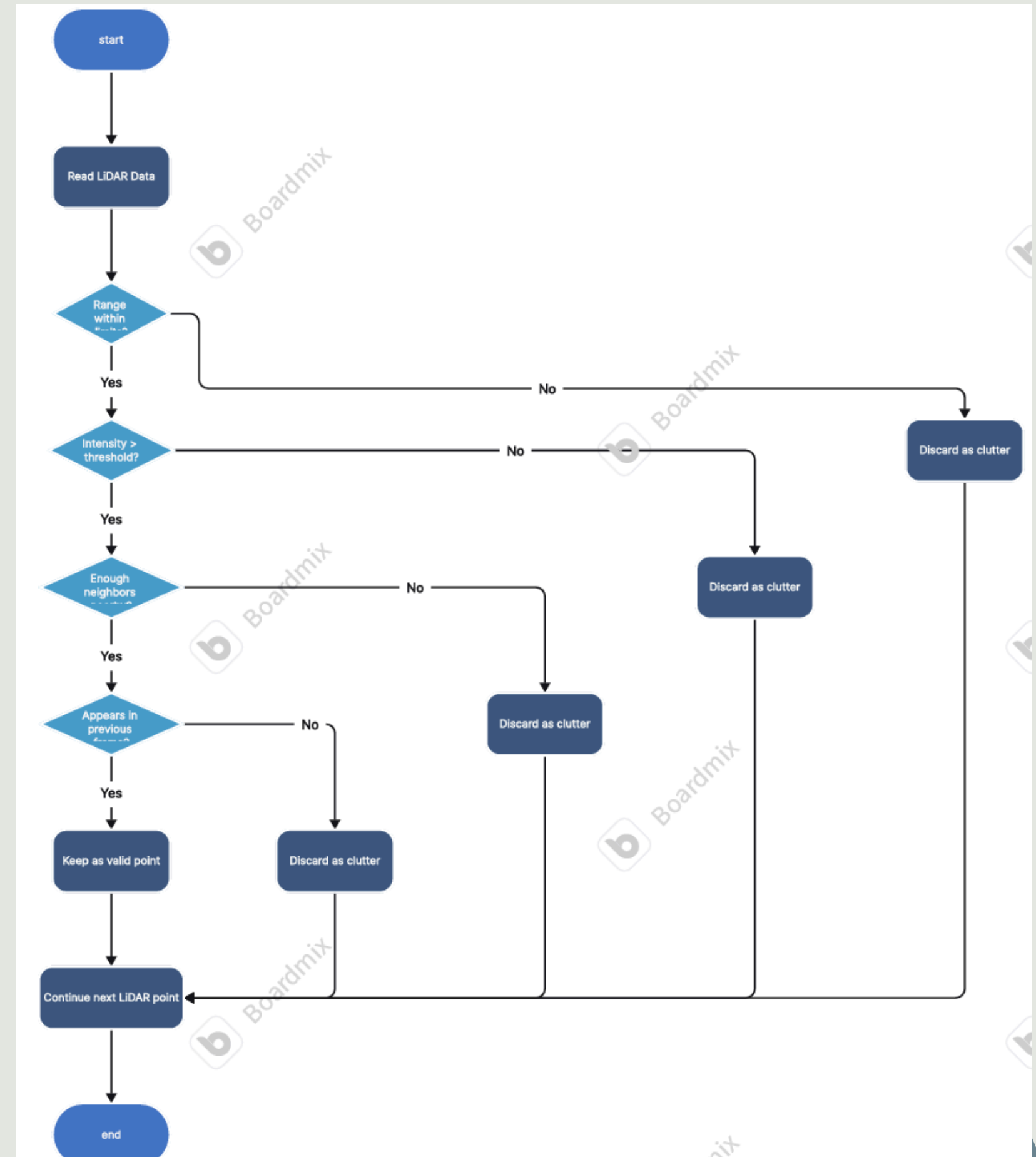
```
    // 4. Temporal stability check
```

```
    if (!exists_in_previous_frame(point, tolerance = 0.1m))  
        continue // not stable over time
```

```
    // 5. Otherwise, keep as valid
```

```
    add_to(clean_points, point)
```

```
return clean_points
```



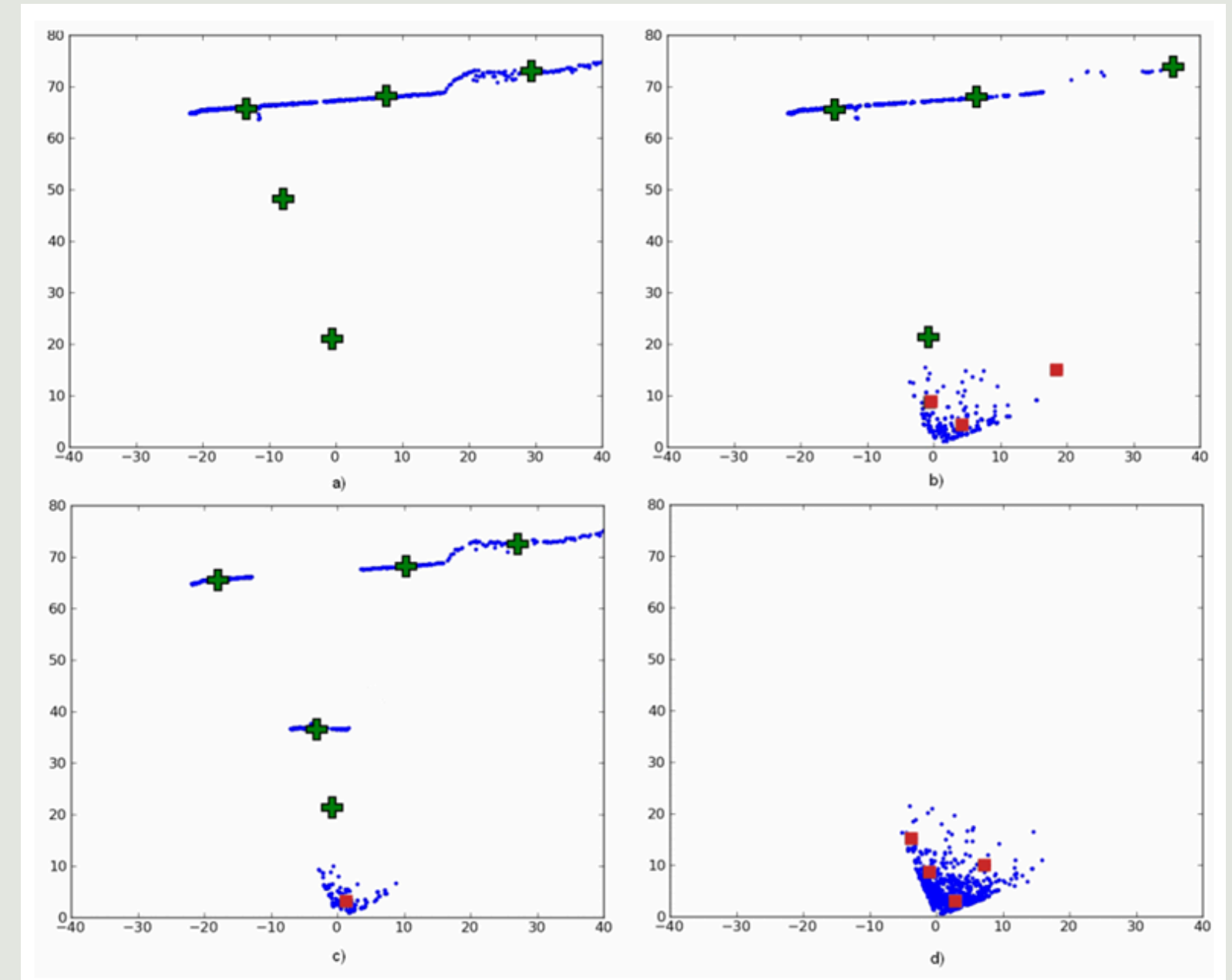
OBSERVATIONS

Four LiDAR frames show the clutter suppression filter in action.

Green crosses mark true targets, red squares mark clutter.

A stationary lamp post appears clearly in frames (a–c), while readings at $(-7, 48)$ and $(-4, 36)$ correspond to vehicles—the latter likely a bus or truck.

In frame (d), heavy snowfall causes almost complete signal loss, leaving only noise



CLUTTER SUPPRESSION FILTER MATLAB SIMUALTION

```
%% improved_clutter_suppression_no_toolbox.m
```

```
clear; clc; close all;
```

```
% --- Simulated LiDAR data ---
```

```
rng(0);
```

```
N = 500;
```

```
X = 100*rand(N,1)-50;
```

```
Y = 100*rand(N,1)-50;
```

```
Intensity = rand(N,1);
```

```
clutter_idx = randperm(N,120);
```

```
Intensity(clutter_idx) = Intensity(clutter_idx)/10;
```

```
figure('Name','Raw LiDAR Data');
```

```
scatter(X,Y,20,Intensity,'filled');
```

```
title('Raw LiDAR Data'); xlabel('X'); ylabel('Y'); axis equal; grid on;
```

```
% --- Parameters (tune these) ---
```

```
TH_intensity = median(Intensity) + 0.25*std(Intensity);
```

```
r_neighbor = 6; % neighborhood radius
```

```
min_neigh = 3; % minimum neighbors required
```

```
valid = false(N,1);
```

```
% --- Manual neighbor counting (no toolbox) ---
```

```
for i = 1:N
```

```
    dx = X - X(i);
```

```
    dy = Y - Y(i);
```

```
    dist = sqrt(dx.^2 + dy.^2);
```

```
    neigh = sum(dist < r_neighbor) - 1; % exclude self
```

```
    if (Intensity(i) >= TH_intensity) && (neigh >= min_neigh)
```

```
        valid(i) = true;
```

```
    end
```

```
end
```

```
% --- Results ---
```

```
Xf = X(valid); Yf = Y(valid); If = Intensity(valid);
```

```
fprintf('\nTotal points: %d\nKept after filter: %d\nRemoved: %d\n',
```

```
...
```

```
    N, sum(valid), N - sum(valid));
```

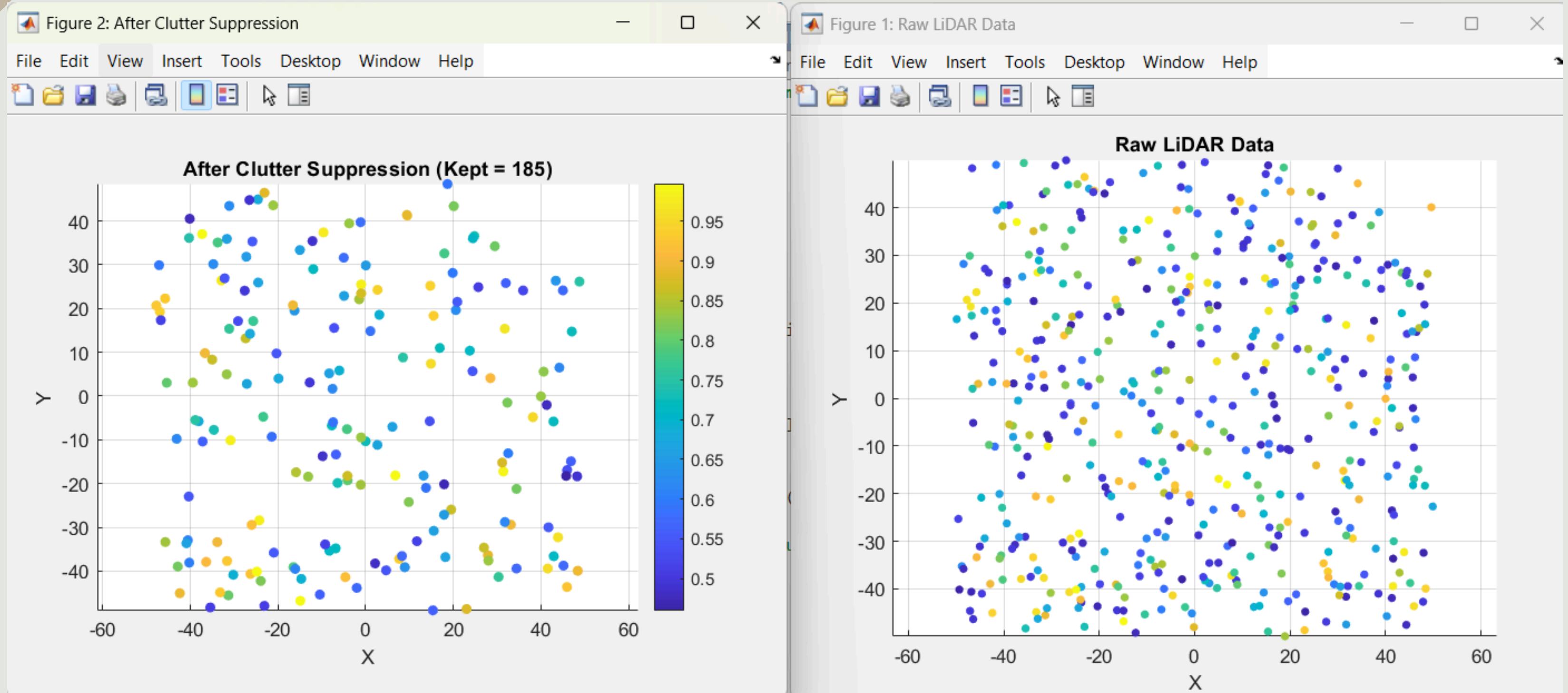
```
figure('Name','After Clutter Suppression');
```

```
scatter(Xf,Yf,30,If,'filled');
```

```
title(sprintf('After Clutter Suppression (Kept = %d)', sum(valid)));
```

```
xlabel('X'); ylabel('Y'); axis equal; colorbar; grid on;
```

SIMUALTION RESULTS



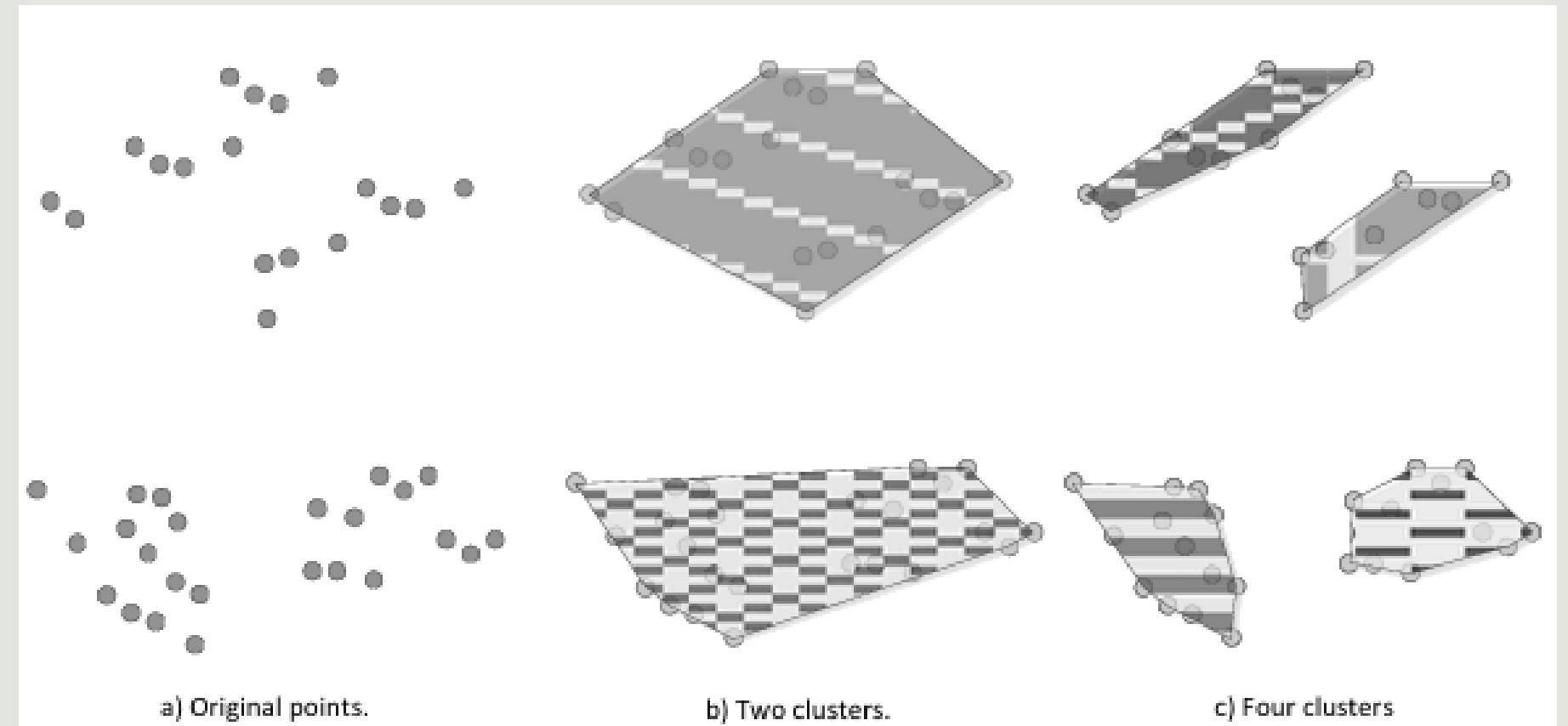
CLUSTERING

Clustering is the process of dividing data with similar properties into discrete groups. The aim is to take advantage of natural properties in the data such as the grouping tendency towards a multi variant normal distribution

The greater the homogeneity of the data within a cluster, the greater is the chance of successful clustering.

The advantages of clustering data are many. These include:

1. Complexity reduction: You can reduce the number of parameters to describe the data. For example, a set of 1,000 points in the Cartesian coordinate system may only need ten clusters to describe its fundamental outlook.
2. Intuitiveness: The human eye have no problem spotting clusters in natural environments. For instance, we can easily distinguish a flock of birds from another even though the flock is made up of hundreds of individuals.



THE BASIC IDEA BEHIND CLUSTERING. A) SHOWS THE ORIGINAL 2D POINTS. IN B), TWO CLUSTERS WHERE CREATED, WHILE IN C) FOUR CLUSTERS WHERE CREATED. AGAIN, THIS SHOWS THE NEED FOR ASSUMPTIONS ABOUT THE SIZE OF THE OBJECTS BEING CLUSTERED

COMMON CLUSTERING METHODS

K-Means Clustering

- Recursive, prototype-based clustering algorithm
- Starts with an initial estimate of cluster centroids
- Partition n data points into k predefined clusters
- Requirement: Number of clusters (k) must be known beforehand
 - Can be an advantage (controlled grouping) or a drawback (may not reflect actual data)
- Risk: Algorithm can get stuck in local minima due to random initialization
- Working steps:
 - a. Randomly select k data points as initial centroids
 - b. Assign each data point to the nearest centroid
 - c. Compute new centroids as the mean of points in each cluster
 - d. Repeat assignment and update steps until convergence (no significant centroid change)

$$\operatorname{argmin}_S \sum_{i=1}^k \sum_{x_j \in S_i} \|x_j - \mu_i\|^2$$

Monte Carlo extended k-Means

- The Monte Carlo algorithm works by running m test series with the k-Means algorithm, generating m different cluster sets. The cluster set with the lowest cost is then selected as the final cluster set. The cost function

$$c_l = \frac{1}{n} \sum_{i=1}^n E(S_i)^2$$

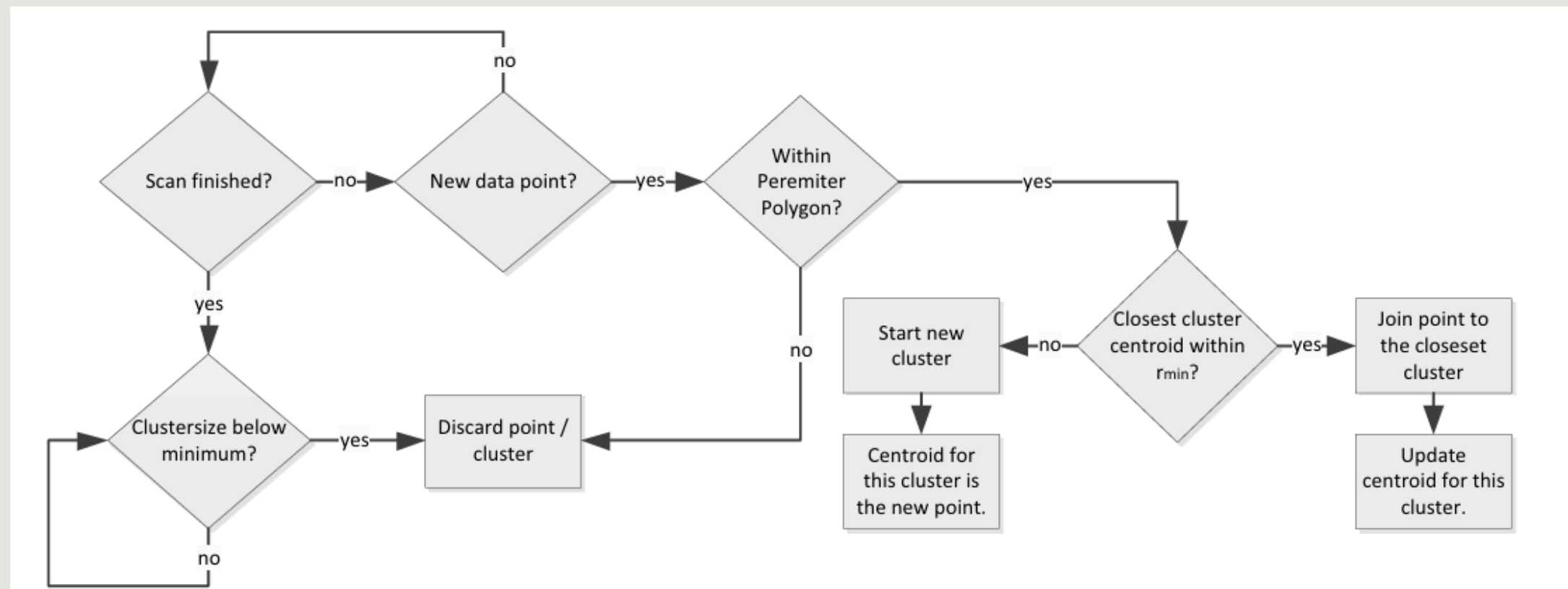
- estimates the mean variance of the cluster set. The theory being that a “good” cluster is more compact than a “bad” cluster and hence have lower variance.

COMMON CLUSTERING METHODS

Online average based clustering

- Traditional clustering methods like k-Means update clusters only after a full LiDAR scan, making them too slow for real-time use and requiring a preset number of clusters.
- This online algorithm creates and updates clusters as data arrives, using the Nearest Neighbor rule:
- If a new point lies within radius $r_{\min}$ of any cluster center $\rightarrow$ assign to that cluster.
- Otherwise $\rightarrow$ form a new cluster dynamically.
- we first have to adapt the way a point is assigned to a cluster. The simplest and most naïve approach of doing this is to use the Nearest Neighbor (NN) technique. If a data point is within a radius to any cluster mean, the point will be assigned to the closest cluster. If not, a new cluster will be formed.
- Cluster means are updated incrementally using:

$$\arg \min_j x_{i,j} = \begin{cases} \arg \min_{0 \leq j \leq k} (\mu_j - x_i)^2 & \text{if } \forall \mu_j: (\mu_j - x_i)^2 \leq r_{\min}^2 \\ k + 1 & \text{otherwise} \end{cases}$$



MATLAB SIMULATION

```
%% Cluster Recognition – K-Means and Online Average
```

```
clear; clc; close all;
```

```
% --- Generate filtered LiDAR-like points ---
```

```
rng(1);
```

```
X = [randn(100,1)*2+10; randn(100,1)*3-20; randn(100,1)*4+25];
```

```
Y = [randn(100,1)*2+10; randn(100,1)*3+15; randn(100,1)*4-20];
```

```
points = [X Y];
```

```
figure; scatter(X,Y,20,'filled'); title('Input LiDAR Points'); axis equal; grid on;
```

```
%% --- A) K-Means Clustering ---
```

```
k = 3;
```

```
idx_kmeans = kmeans(points, k);
```

```
figure; gscatter(X,Y,idx_kmeans); title('K-Means Clustering'); axis equal; grid on;
```

```
%% --- B) Online Average-Based Clustering ---
```

```
r_min = 8; % clustering radius
```

```
clusters = [];
```

```
centers = [];
```

```
cluster_count = 0;
```

```
for i = 1:size(points,1)
```

```
    p = points(i,:);
```

```
    if isempty(centers)
```

```
        cluster_count = 1;
```

```
        centers(1,:) = p;
```

```
    else
```

```
        else
```

```
            dist = sqrt(sum((centers - p).^2,2));
```

```
            [dmin, idx] = min(dist);
```

```
            if dmin < r_min
```

```
                clusters{idx} = [clusters{idx}; p];
```

```
                n = size(clusters{idx},1);
```

```
                centers(idx,:) = centers(idx,:) + (p - centers(idx,:)) / n;
```

```
            else
```

```
                cluster_count = cluster_count + 1;
```

```
                centers(cluster_count,:) = p;
```

```
                clusters{cluster_count} = p;
```

```
            end
```

```
        end
```

```
    end
```

```
% Plot Online Average clusters
```

```
figure; hold on;
```

```
colors = lines(cluster_count);
```

```
for i = 1:cluster_count
```

```
    pts = clusters{i};
```

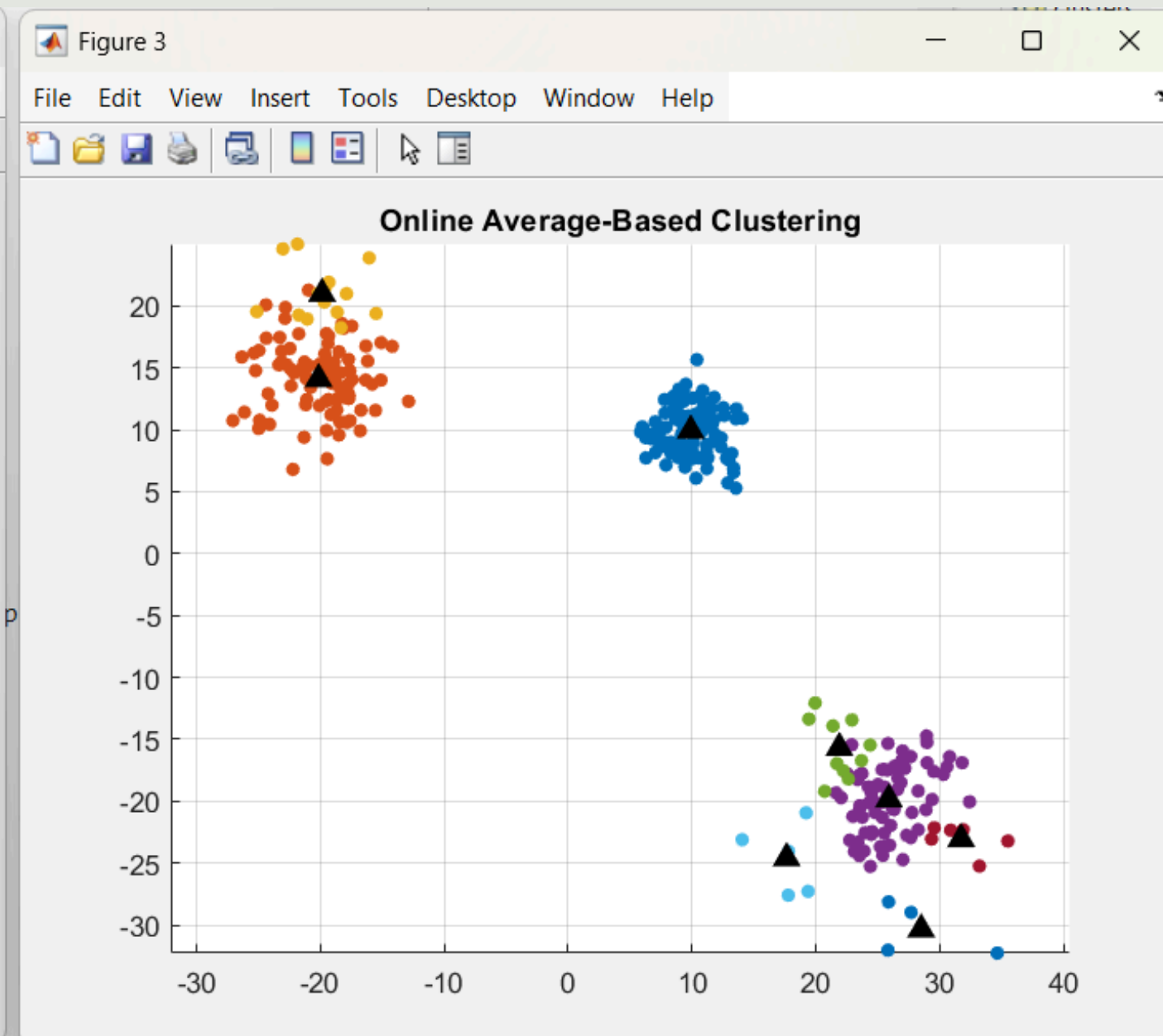
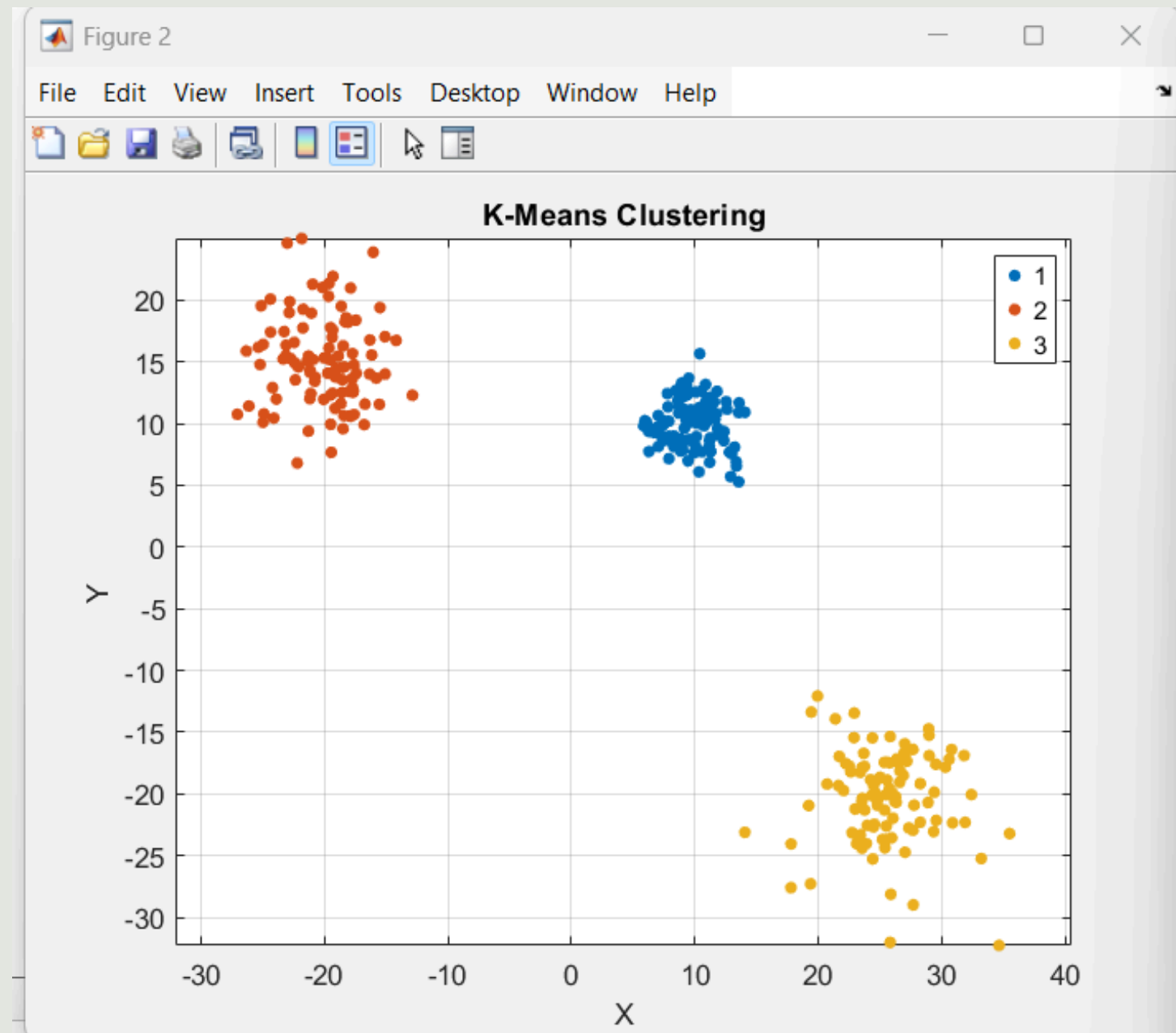
```
    scatter(pts(:,1), pts(:,2), 25, colors(i,:), 'filled');
```

```
end
```

```
scatter(centers(:,1), centers(:,2), 80, 'k^', 'filled');
```

```
title('Online Average-Based Clustering'); axis equal; grid on;
```

MATLAB SIMULATION



POLYGON FITTING

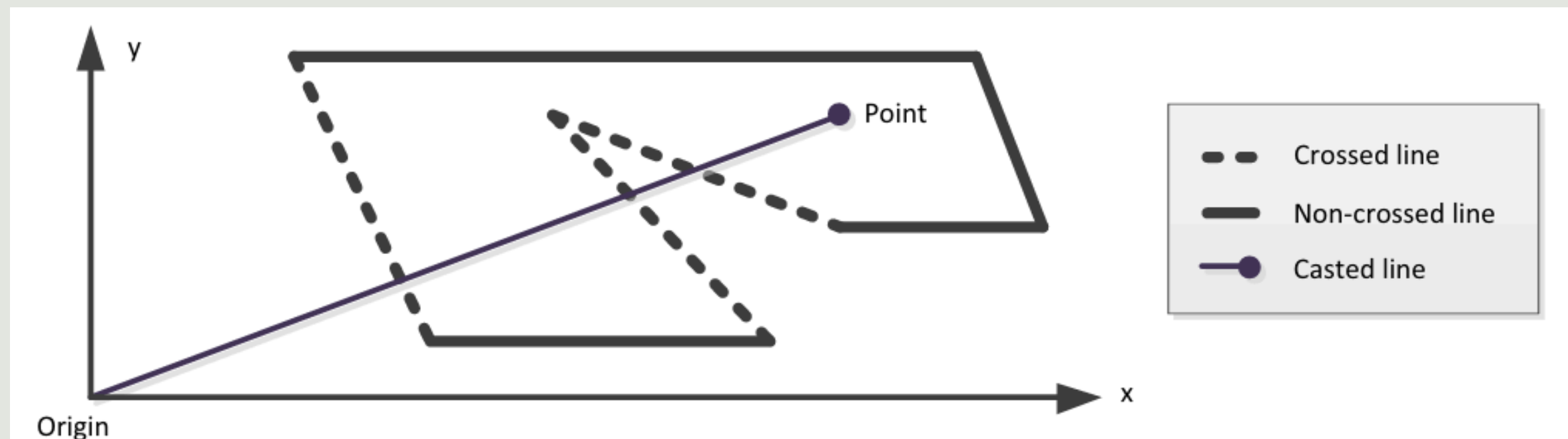
When a LiDAR scans, it collects points from everything around it — the road, nearby buildings, trees, sky reflections, etc.

But often, we're only interested in a specific region — for example, the road area in front of the car

To avoid clustering unwanted background points, a Perimeter Polygon is defined — only LiDAR points inside this polygon are considered relevant for processing.

The Ray Casting Algorithm is used to test if a point lies inside or outside the polygon:

- Imagine a ray drawn from the point to outside the polygon.
- Count how many times the ray intersects the polygon's edges.
- If the count is odd → point is inside,
- if even → point is outside.



MATLAB SIMULATION POLYGON FITTING

```
%% Polygon Filtering (Closed Polygon)
```

```
clear; clc; close all;
```

```
% --- Simulated clustered points ---
```

```
rng(5);
```

```
X = 100*rand(200,1)-50;
```

```
Y = 100*rand(200,1)-50;
```

```
% Define polygon (ROI)
```

```
polyX = [-30 30 30 -30 -30]; % added last point same as first
```

```
polyY = [-10 -10 50 50 -10]; % added last point same as first
```

```
% Find points inside polygon
```

```
in = inpolygon(X, Y, polyX, polyY);
```

```
% --- Plot Results ---
```

```
figure; hold on; axis equal; grid on;
```

```
plot(polyX, polyY, 'k--', 'LineWidth', 1.5); % closed polygon
```

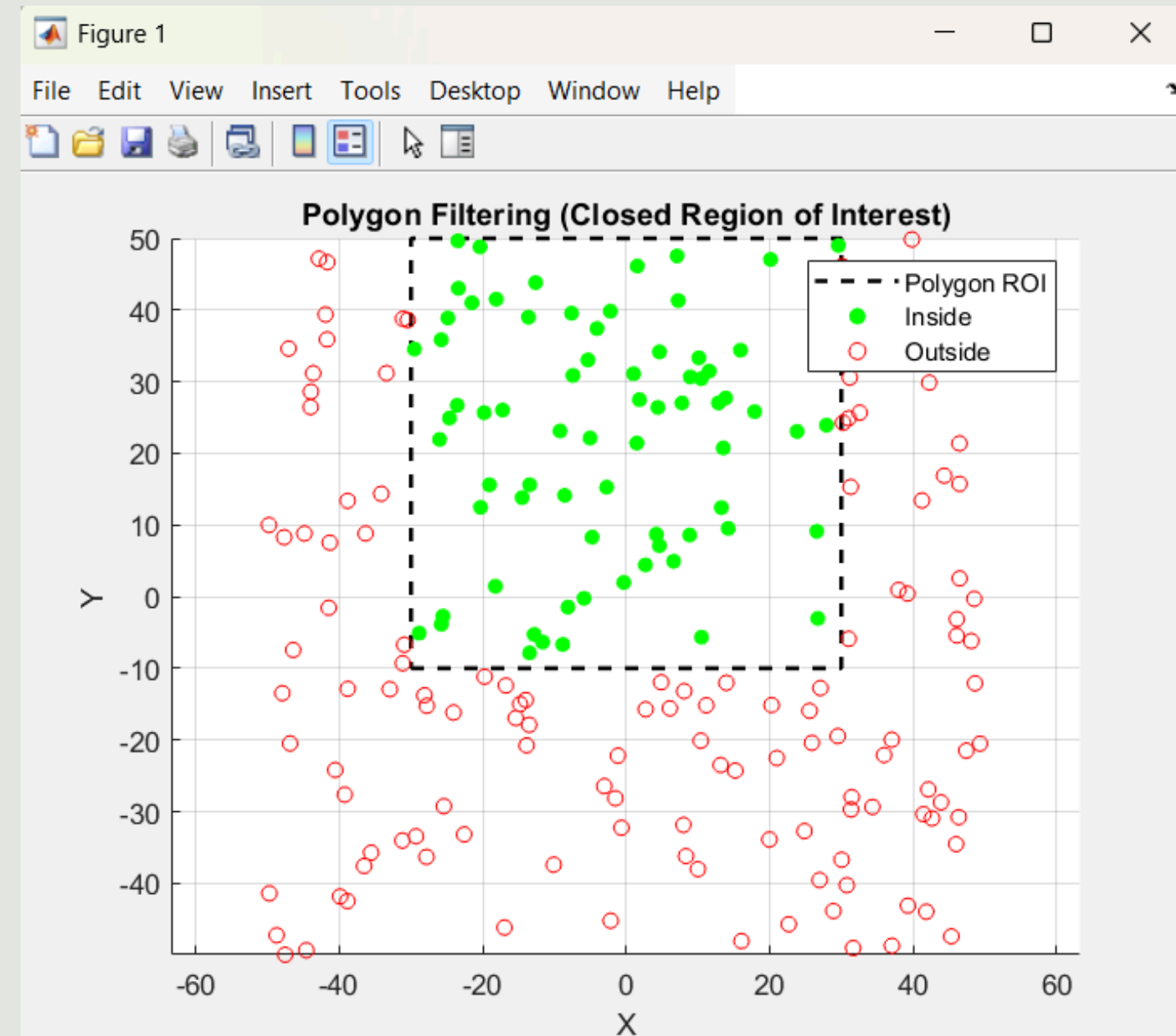
```
scatter(X(in), Y(in), 30, 'g', 'filled'); % inside polygon
```

```
scatter(X(~in), Y(~in), 30, 'r'); % outside polygon
```

```
legend('Polygon ROI','Inside','Outside');
```

```
title('Polygon Filtering (Closed Region of Interest)');
```

```
xlabel('X'); ylabel('Y');
```



DEPTH CAMERA

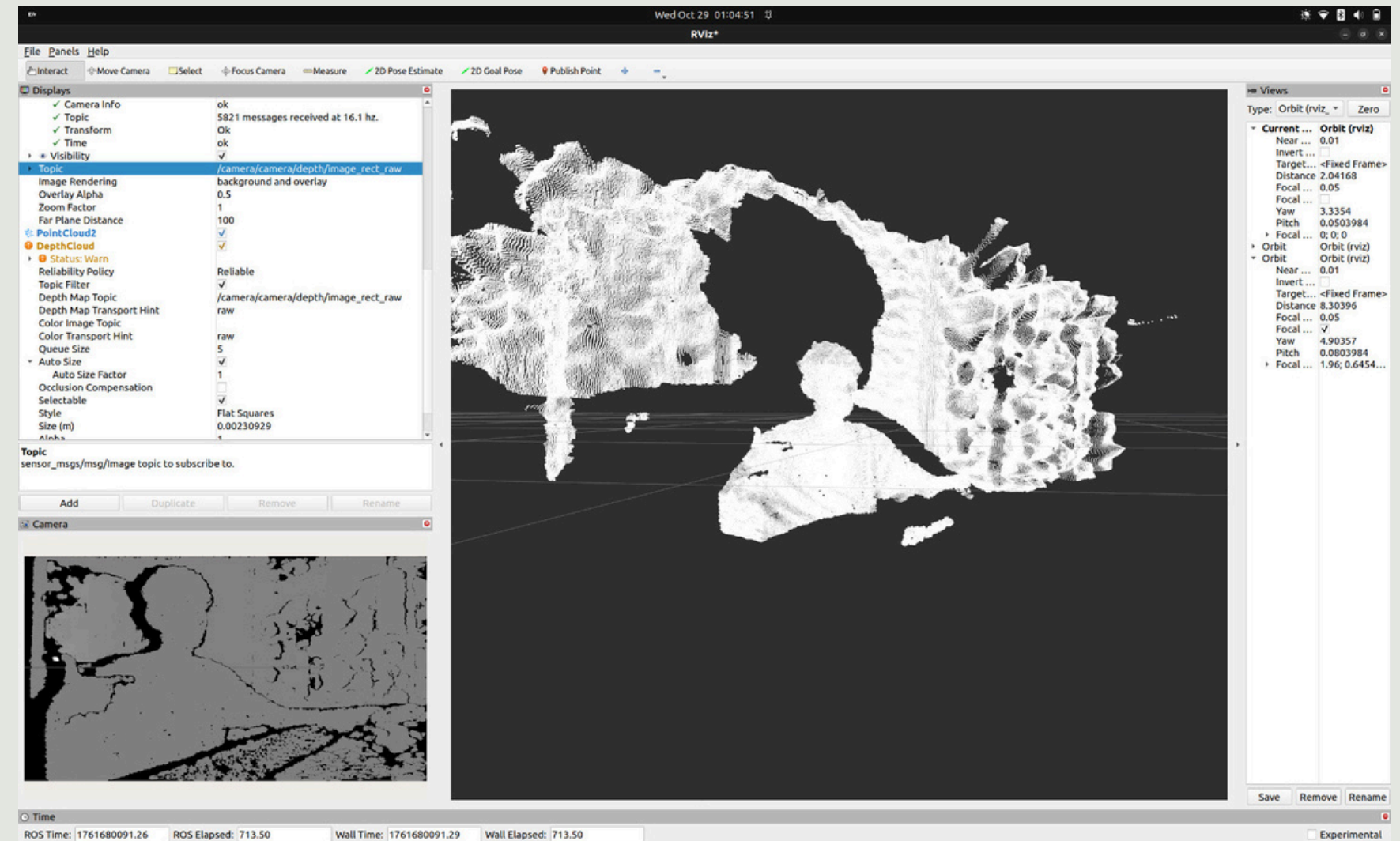
Intel RealSense D435i

Key Features

- Stereo Depth Sensing: Dual infrared sensors + RGB sensor generate accurate 3D depth maps.
- IMU (Inertial Measurement Unit): Built-in accelerometer & gyroscope for motion tracking and sensor fusion.
- Wide Field of View (FOV): $\sim 86^\circ \times 57^\circ$, ideal for robotics and autonomous navigation.
- Depth Range: Up to 10 meters, with sub-millimeter precision at close range.
- High Frame Rate: Supports up to 90 FPS for real-time depth streaming.
- Compact & Lightweight: Easily mountable on vehicles, drones, and robots.

Working Principle

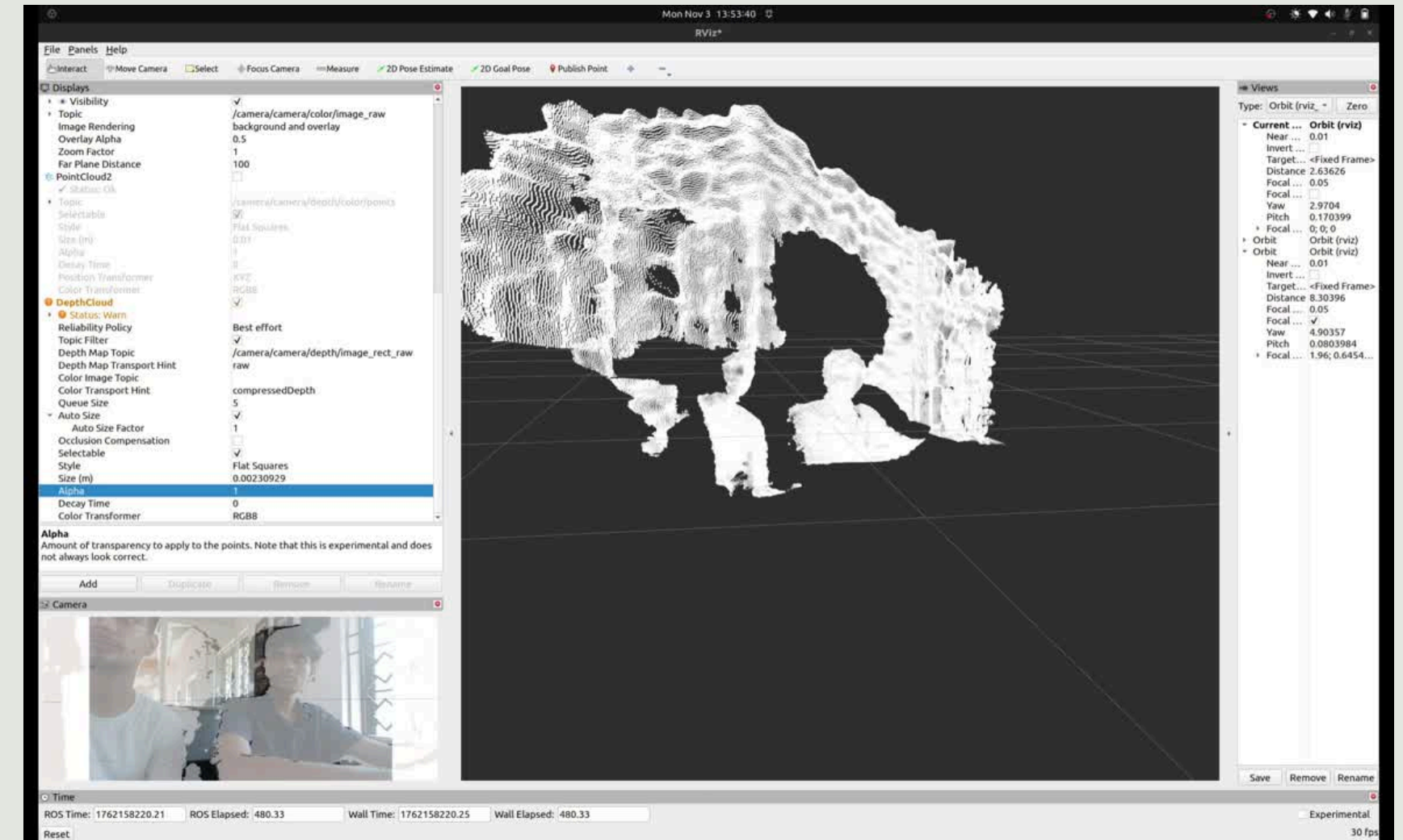
- Uses stereo vision: two IR cameras calculate depth via disparity between left and right images.
- IR projector helps in low-texture or low-light conditions by adding structured patterns.



DEPTH CAMERA

Advantages over 2D LiDAR / RGB Cameras

- Provides 3D spatial data (X, Y, Z), enabling true object shape recognition.
- Captures color + depth, allowing semantic and geometric understanding.
- Real-time clustering becomes more robust (distance thresholding in 3D).
- Improved clutter suppression: Distinguish background using depth cutoff.
- Dynamic Region of Interest (ROI): Polygon or 3D bounding box filtering based on depth.
- Supports SLAM & Odometry: IMU + depth integration enhances localization accuracy.
- Works both indoors and outdoors under varying lighting conditions.



[video link](#)

MOTOR ENCODER - SIGNAL CONDITIONING

Encoders (optical or magnetic) on the motor shaft or wheels provide precise rotational speed and direction, which is crucial for:

- Odometry — estimating vehicle displacement and orientation.
- Feedback control — ensuring the motors follow trajectory commands.
- Low-level navigation — correcting wheel slip or drift detected during path following

In autonomous vehicles, path planning requires knowing the vehicle's exact position, orientation, and velocity.

Here, for our use case, we will be using **Magnetic Incremental Encoder**

WHY DO WE NEED SIGNAL CONDITIONING : These signals are low voltage (5V TTL or 3.3V logic) and automotive environments are usually noisy.

SIGNAL CONDITIONING SCHEMATIC:

Encoder → Line Receivers → RC Filter → Schmitt Trigger → Microcontroller



MOTOR ENCODER - ALGORITHM IMPLEMENTATION FLOWCHART

LOW LEVEL ENCODING

PRJC Encoder Lib

VELOCITY ESTIMATION

α - β Filter / FilterLib

NOISE REJECTION

Welford + Hampel

STATIC CALIBRATION

TinyMatrix least-squares

DYNAMIC COMP

Recursive LS

SMOOTHING

Savitzky-Golay / EMA



REAL-TIME SIGNAL PROCESSING ALGORITHM

The following is the magnetic incremental encoder processing pipeline and the suggested standard algorithm/library/reference implementation for it (Full Arduino Code in Report):

1. **Quadrature decoding (ISR-based)** - PJRC Encoder Library (Encoder.h) - Extremely robust ISR-based quadrature decoding, supports up to 12 encoders; uses optimized attachInterrupt handlers. Handles A/B phase transitions, direction, and index automatically.
2. **Real-time velocity and position estimation** - Derivative of Encoder counts + Alpha-Beta Filter

```
#include <Filter.h> // from JonHub's FilterLib

AlphaBetaFilter<float> abFilter(0.5, 0.1); //  $\alpha$ ,  $\beta$  tuned

void updateVelocity(float measuredPos, float dt) {
    abFilter.update(measuredPos, dt);
    float pos_est = abFilter.getPosition();
    float vel_est = abFilter.getVelocity();
}
```

REAL-TIME SIGNAL PROCESSING ALGORITHM

3. Statistical analysis (mean, variance, outlier

rejection) -Welford's online algorithm for mean & variance + Hampel Filter for outlier rejection - Welford gives numerically stable streaming statistics; Hampel filter replaces outliers using median $\pm k\sigma$.

```
// --- Welford online mean/variance ---
void updateStats(float x) {
    n++;
    double delta = x - mean;
    mean += delta / n;
    M2 += delta * (x - mean);
}

float variance() { return (n > 1) ? M2 / (n - 1) : 0.0; }

// --- Hampel outlier filter ---
float hampelFilter(float* data, int size, int k, float t0) {
    // window k, threshold t0 * median absolute deviation
    // simple implementation from open-source reference
}
```

4. Static & dynamic error compensation -

Polynomial calibration (static) + Dynamic compensation model implemented via Recursive Least Squares (RLS) - RLS allows online estimation of dynamic coefficients (speed-dependent errors). For static calibration, polynomial coefficients come from least-squares fit (see below).

```
// Using RLS filter to estimate dynamic coefficients
#include <RLSFilter.h>
RLSFilter rls(3, 0.98); // 3 coefficients (d0,d1,d2), forgetting factor λ

void updateDynamicComp(float velocity, float measuredError) {
    Eigen::VectorXd phi(3);
    phi << 1.0, velocity, velocity*velocity;
    double corr = rls.update(phi, measuredError); // online coefficient update
}
```

REAL-TIME SIGNAL PROCESSING ALGORITHM

5. Polynomial fitting (least-squares quadratic)

TinyMatrix or uLAPACK + OLS algorithm (ordinary least squares) - use micro linear algebra lib to compute $(X^T X)^{-1} X^T y$ for quadratic fit. Much cleaner & testable.

```
#include <TinyMatrix.h>
using namespace TinyMatrix;

Matrix<double, N, 3> X; // [1, x, x²]
Matrix<double, N, 1> y;
Matrix<double, 3, 1> beta;

beta = (X.transpose() * X).inverse() * X.transpose() * y;
```

6. Smoothing / filtering and pulse shaping

Savitzky-Golay Filter (for smoothing derivatives) OR 1st-order IIR low-pass (EMA) OR Alpha-Beta Filter - Savitzky-Golay is a classic for smoothing noisy discrete signals while preserving shape. EMA (IIR) is simpler for real-time. Use whichever MCU performance allows.

```
#include <SavitzkyGolay.h>
SavitzkyGolay sgFilter(5, 3); // 5-pt window, 3rd order poly

void loop() {
    float smoothedVel = sgFilter.update(rawVel);
}
```

Simulations



[video link](#)



Protocols

DATA DISTRIBUTION SERVICE (DDS)

- ROS 2 does not use a simple protocol like MQTT for its core communication. It is built directly on DDS (Data Distribution Service), an industrial-grade standard for real-time, high-performance systems.

What is DDS?

- It is the native, default middleware for all ROS 2 topics, services, and actions.
- It is a decentralized (peer-to-peer) system. There is no central "master" node (like roscore in ROS 1), making it more robust.
- Nodes find each other automatically using a "Discovery" process.
- The RMW (ROS Middleware Layer) ROS 2 is flexible. It uses an abstraction layer called the RMW that allows you to "plug in" different DDS vendors:
 - Default (Humble): eProsima Fast DDS
 - Other Options: Eclipse Cyclone DDS, RTI Connex, etc.
- Key takeaway: The protocol for ROS 2 pub/sub is DDS. The underlying wire protocol is RTPS (Real-time Publish Subscribe)

THE NETWORK STACK: HOW DDS (ROS 2) USES WI-FI

- Understanding the Layers Wi-Fi (IEEE 802.11) is not the communication protocol for ROS 2; it is the physical network that carries the protocol's messages.
- Analogy: The Postal Service
 - Your Data (e.g., LiDAR scan): The content of your letter.
 - DDS (RTPS): Puts the letter in an envelope and defines the rules of mail.
 - UDP (User Datagram Protocol): The mail service you chose (e.g., standard post—it's fast, but no delivery confirmation).
 - IP (Internet Protocol): The "address" on the envelope.
 - Wi-Fi (IEEE 802.11): The "mail truck and roads" that physically move the envelope.
- The ROS 2 Stack (Simplified):
 - ROS 2 Node (Application)
 - DDS / RTPS (Middleware)
 - UDP (Transport Protocol)
 - IP (Network Protocol)
 - Wi-Fi / IEEE 802.11 (Physical Network)

KEY ROS 2 FEATURES: QOS & SECURITY

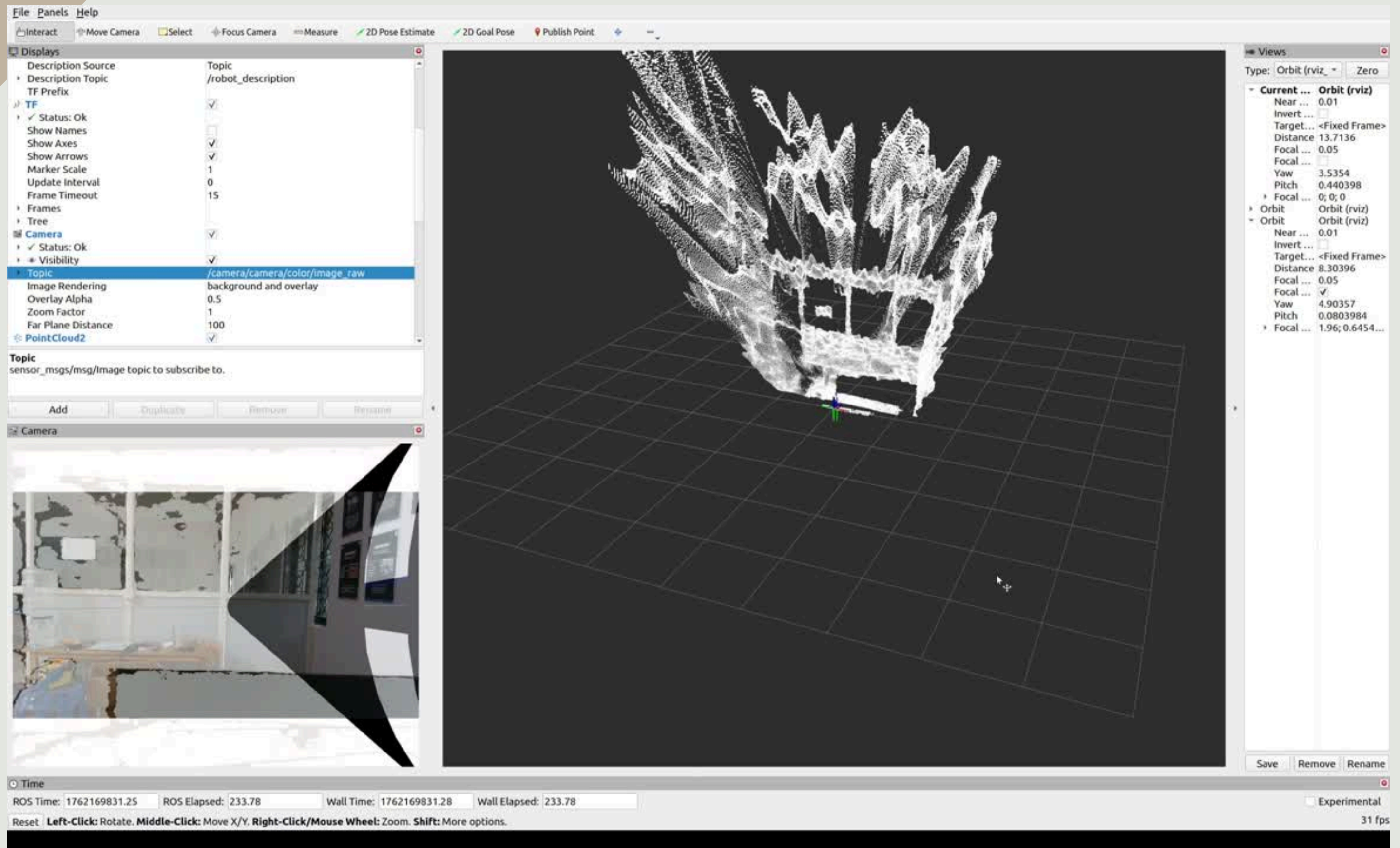
- Tunable Quality of Service (QoS) ROS 2 (via DDS) allows you to configure how data is sent for each topic.
- For /scan & /cmd_vel (Real-time):
 - Reliability: BEST_EFFORT (It's better to drop old packets than to be late).
- For /map & /robot_description (Critical Data):
 - Reliability: RELIABLE (Must guarantee delivery).
 - Durability: TRANSIENT_LOCAL (New nodes get the last sent map).
- A Layered Security Model Security is handled at two levels:
 - Network-Level (Wi-Fi):
 - Protocol: WPA2 / WPA3
 - Job: Secures the entire wireless link from outsiders.
 - ROS 2-Level (DDS):
 - Protocol: DDS-Security
 - Job: Secures the internal ROS 2 graph with node authentication, access control, and per-topic encryption.



Hardware



[video link](#)



[video link](#)